

Creature Care Game

Game Developer Challenge

THE BRIEF

We want to see how you approach building a small game from scratch. Your task is to build a creature care game, a living pet exists on screen, has needs, and reacts to how well (or poorly) it is looked after. The design, personality, and world it lives in are entirely up to you.

This is an open brief. The concept is fixed. But the design, personality, mechanics, and implementation are entirely yours. We want to see your thinking, your craft, and your instincts as a game developer.

CORE REQUIREMENTS

The following mechanics must be present in your submission:

The Pet

Mechanic	What we expect
At least 3 stats	Choose your own names — hunger, happiness, energy, hygiene, or anything that fits your pet's personality. Each stat should feel meaningful.
Stats decay automatically over time	Stats should drop on their own without player input. The rate of decay is up to you — balance it so the game feels challenging but not impossible.
At least 3 player actions	For example: feed, play, sleep. Each action should affect one or more stats. Actions should feel purposeful, not arbitrary.
Visual representation on screen	The pet must always be visible. Style and form are entirely your choice.

Pet States & Feedback

Mechanic	What we expect
At least 3 pet states	For example: happy, sad, sick, sleeping, critical. States should change based on current stat levels and be immediately readable to the player.
Visual or audio feedback on state change	The player must always know when something changes. This could be a sprite change, animation, sound, colour shift, or any combination.
Game over condition	If the player neglects the pet long enough, it should die. The death state must be clearly communicated — not just a freeze or blank screen.

Mechanic	What we expect
Stat indicators always visible	The player should be able to read all stat levels at a glance at any point during gameplay — bars, icons, numbers, or any clear display.

Game Loop

Mechanic	What we expect
Real-time stat decay	The game runs continuously. Stats should decay over time regardless of whether the player is actively doing something. Use a timer or coroutine — not turn-based logic.
Actions have a cooldown or cost	Prevent the player from spamming a single action to keep all stats maxed. A cooldown timer, energy cost, or similar mechanic should make the game require actual attention.
A clear goal or survival condition	The player needs something to aim for. This could be keeping the pet alive for a set number of days, reaching a milestone, or simply surviving as long as possible.

BONUS (OPTIONAL)

These are not required but will demonstrate extra polish and initiative:

- **Save / load state** — pet persists between sessions (local storage, file, or equivalent)
- **Day / night cycle** — time of day affects stat decay rates or available actions
- **Name your pet** — a name input screen at the start that personalises the experience
- **Sound effects or music** — audio feedback for actions and state changes
- **Pet age / lifecycle** — pet grows or evolves over time based on care quality

TECHNICAL NOTES

Engine

Unity is the preferred engine. If you have a strong reason to use a different engine, you may do so but you must justify the choice in your written document.

Art style

2D is the preferred art style. You are free to use any 2D visual approach — sprites, shapes, UI-driven, or anything else.

Use of AI tools

You may use AI-assisted tools as part of your workflow. However, this challenge is designed to assess your game development skills, design thinking, and implementation quality — not your ability to prompt an AI. Heavy reliance on AI-generated code without a clear understanding or the ability to justify your decisions will negatively impact your evaluation. We're hiring you, not the AI.

DELIVERABLES

- **Source code** — GitHub repo (preferred), or zipped via email.
- **Playable build** — an executable or a WebGL / browser build (itch.io, GitHub Pages, etc.) is strongly preferred.
- **Setup instructions** — clear enough that we can run or play it without asking you anything.
- **A short-written document** — engine choice, design decisions, what you would add or improve with more time.

WHAT WE LOOK FOR

We care about how your game feels to play, how you structured your code, and how you made decisions under time pressure.

A small, polished, and fun game will always score higher than a feature-heavy but broken one.